

lqcddb 蒸馏收缩框架核心部分穷尽剖析

opencode pure

2026 年 8 月 15 日

摘要

本报告对 `refer/sush` 下的 `lqcddb` 蒸馏 (distillation) 关联函数工具包进行穷尽式核心剖析。项目性质判定为 **代码-物理交叉向**：核心是”从强子算符字符串自动生成 Wick 缩并图并执行张量收缩”的物理算法链，每个代码符号与格点 QCD 物理对象一一对应。共枚举 12 个核心部分候选 (**5 深挖 / 6 浅析 / 1 排除**)，深挖对象为：自动 Wick 收缩引擎、等价图识别 (并查集 + BFS 符号传播)、动态收缩工作流 (三注册表 + 计划缓存)、Roofline 带宽分析器、缓存 einsum 收缩入口。最强竞争力：以 **算符字符串 $\rightarrow$ einsum 表达式** 的全自动降维，把 Wick 收缩这一格点 QCD 传统手工易错环节化为纯符号算法，配合等价图归并与缓存缩并将计算量降至最低。

目录

1	项目定位与核心部分清单	2
1.1	项目性质判定	2
1.2	项目背景与两套代码	2
1.3	核心部分总清单 (穷尽枚举)	2
2	核心部分深度剖析	3
2.1	C1 自动 Wick 收缩引擎 (<code>wick_contraction</code>)	3
2.2	C2 等价图识别 (<code>identify_equivalent_diagrams</code>)	5
2.3	C3 动态收缩工作流 (<code>dynamic.py</code>)	5
2.4	C4 Roofline 带宽分析器 (<code>contractadviser.py</code>)	6
2.5	C5 缓存 einsum 收缩入口 (<code>cached_contract</code>)	7
2.6	C6-C11 浅析 (一段式概述)	7
3	独特性与竞争力评估	8
4	关键片段与公式	8
4.1	核心代码片段 (<code>lstinputlisting</code> 直引)	8
4.2	核心公式汇总	15
5	参考源清单	15
6	结论	16

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质: 代码-物理交叉向。依据 (实测): lqcddb/src 下 Python 源码 11669 行, 其中 contraction/ 核心收缩机制 4147 行 (占 35%), base/ 基础层 1900 行, analyse/ 统计分析 914 行, eigvectors/ 顶点与本征矢 1090 行; 代码主体即物理计算 (Wick 收缩、蒸馏顶点、传播子缩并), 且算符字符串、缩并指标、顶点对象均直接对应物理量——符号层与物理层高度耦合, 无独立公式文档 (physics.md 为算符库/参数表, 非推导文档)。

1.2 项目背景与两套代码

refer/sush 下并存两套代码: 顶层扁平模块 function_contraction/ (5892 行, 原始版本) 与正式打包的 lqcddb/ (pip install -e ., 积极开发中)。两者暴露相同核心 API (wick_contraction、Jackknife、meff、set_backend), lqcddb 为前者重构版: 拆分模块、增加懒加载、缓存缩并、动态收缩注册表与带宽顾问。本报告以 lqcddb 为主体剖析, 旧模块仅作对比基准与排除项处理。

1.3 核心部分总清单 (穷尽枚举)

编号	核心部分	处理态	独特性概述
C1	autowick.py 自动 Wick 收缩引擎	深挖	从强子算符字符串自动穷举所有合法缩并图、Fermi 符号、VVV/VDV 顶点与 einsum 指标串
C2	autowick.py 等价图识别	深挖	并查集 + BFS 符号传播识别等价缩并图, 消除冗余计算并正确累计相对符号
C3	dynamic.py 动态收缩 workflow	深挖	Peram/V/Gamma 三注册表 + 收缩计划缓存 + 按需计算的高层 workflow
C4	contractadviser.py Roofline 带宽分析	深挖	带宽感知的收缩分析器 (只分析不计算), 给出切分自由指标的建议
C5	base_functions.py 缓存 einsum	深挖	opt_einsum 表达式缓存 + 多种策略自动择优 + 零内存形状校验
C6	analyse.py 统计分析	浅析	Jackknife/Bootstrap/meff/GEVP/3pt-2pt 比值, 向量化协方差构建
C7	eigvectors/vertex.py 顶点函数	浅析	VdV/VVV 动量投影、带规范链接的 VdV、 Ω 权重张量

编号	核心部分	处理态	独特性概述
C8	baroperator.py 算符共轭	浅析	强子算符厄米共轭 (自动识别介子/重子结构、 γ 链 H/T/C 变换)
C9	__init__.py 懒加载 API	浅析	__getattr__ 按需导入具体模块, 避免加载重依赖
C10	mpi_init.py MPI 并行	浅析	时间片轮询分配 + numpy/cupy 混合 MPI 数据搬运
C11	constant/gamma_matrix.py DR 基 γ 矩阵	浅析	DeGrand-Rossi 基 18 种矩阵与动量到 γ 的 Levi-Civita 映射
C12	function_contraction/ 旧扁平模块	排除	被 lqcddb 取代 (同 API 重构版), 仅作参考对比基准

穷尽性说明: 候选按 lqcddb/src 全部 9 个模块 + 旧模块枚举, 三态填满无”未处理”。C12 排除理由: 为旧版 (corr_wick.py 等), 核心逻辑已在 lqcddb 中重构升级, 直接剖析新版即为对旧版的穷尽。

2 核心部分深度剖析

2.1 C1 自动 Wick 收缩引擎 (wick_contraction)

物理图像

物理图像: 给定 s/source/current 三组强子算符字符串 (如 ['|', 'u', 'C*gamma_5', 'd', 'u', '|'] 表示质子), Wick 定理自动将场算符乘积按”同味夸克-反夸克配对”展开为所有可能的缩并图, 每图贡献一个 Fermi 符号 $(-1)^{N_{inv}}$ 与一个蒸馏收缩表达式 (peram $\times \gamma \times$ 顶点, 用 einsum 缩并)。对应公式 (1)。

$$\langle \mathcal{O}_{\text{sink}}(t) J(t') \mathcal{O}_{\text{src}}^\dagger(0) \rangle = \sum_{\{\text{pairings}\}} \varepsilon_\pi \text{Tr}[\gamma \cdot \tau \cdot \gamma \cdots] \times \prod_{\text{vertex}} V(p) \quad (1)$$

算法原理 (输入 $\rightarrow$ 输出 $\rightarrow$ 步骤):

- 输入解析** (autowick.py:239-254): 拼接三组算符, 数字元素 (int/complex/float) 作为全局系数提出; 类型检查。
- 通配符展开** (autowick.py:258-293): 'q' 展开为六味、'l' 展开为轻味 u/d, 笛卡尔积生成所有味替换方案; 味不平衡的替换直接跳过 (autowick.py:307-314)。
- 缩并图穷举** (generate_valid_contractions, autowick.py:175-224): 按味分组, 每味内夸克与反夸克做所有一一配对 ($n_q!$ 种排列), 跨味取笛卡尔积, 得全部合法缩并图。

4. **Fermi 符号** (`compute_fermion_sign`, `autowick.py:156-173`): 对每对 $(q, \bar{q})$ 按算符字符串位置排序后计算逆序数, 符号为 $(-1)^{\text{inv}}$ 。
5. **顶点分配** (`creat_sink_source`, `autowick.py:67-99`): 按 '|' 分隔符划分时间区间, 区间内 3 个夸克 $\rightarrow$ VVV (重子顶点)、2 个 $\rightarrow$ VDV (介子顶点), 顶点指标字母自动取对应夸克收缩标签。
6. **结果生成** (`autowick.py:406-441`): 每图输出 einsum 指标串 `result_indx` (如 'ab,cd,ef->gh')、组件名 `result_name`、Fermi 符号 `result_sign` (乘全局系数)。

核心算法

核心算法: 按味分组的完美匹配枚举 (`itertools.permutations` + 笛卡尔积); 复杂度为 $\prod_{\text{flavor}} n_f!$ 个缩并图, 对角动量 L 的算符串等价于所有指标配对, 图上顶点数 $\leq 2L$; Fermi 符号用逆序数 $O(k^2)$ (k 为场数)。对典型质子 2pt (3 对缩并) 为 $3!$ 个匹配 $\times 2$ 组 = 2 图 (direct + exchange), 与 README.md:132-139 所述一致。未验证: 最大算符规模 (多强子 4pt) 的实测枚举耗时未做基准。

代码-物理映射

映射原则: 每个关键代码符号必有物理对象, 双向可查, 无孤儿符号。

代码符号	物理对象	物理含义与参考源
'u','d','s'...	夸克味场	六味夸克; <code>lqcddb/src/lqcddb/contraction/autowick.py:229</code>
'u~d'	反夸克	反夸克标记; <code>autowick.py:88</code>
' '	强子分隔符	划分时间区间与强子边界; <code>autowick.py:136-144</code>
<code>gamma_5</code> 等	γ 矩阵插入	狄拉克空间插入; <code>autowick.py:332-333</code>
<code>contraction_index</code>	收缩指标	einsum 指标对 (ab, cd, ...); <code>autowick.py:232-233</code>
<code>VVV_i</code>	重子顶点	Levi-Civita 颜色缩并的三夸克顶点; <code>autowick.py:93-95</code>
<code>VDV_i</code>	介子顶点	两夸克顶点; <code>autowick.py:96-98</code>
<code>peram</code>	蒸馏传播子	特征空间传播子; <code>autowick.py:406-416</code>
<code>result_sign</code>	Fermi 符号	Wick 符号 $\times$ 全局系数; <code>autowick.py:417-418</code>
<code>tsink/tsrc/tcur</code>	时间片	<code>sink/source/current</code> 时间标签; <code>autowick.py:357-374</code>

独特竞争力: 传统格点 QCD 中 Wick 收缩靠手工逐图推导 (多图时极易漏图/符号错), 本实现将整个流程符号化自动化, 并直接输出可执行的 einsum 字符串, 与物理表达式一一对应, 杜绝手工缩并错误。

2.2 C2 等价图识别 (identify_equivalent_diagrams)

物理图像

物理图像: 同一关联函数中, 多个 Wick 缩并图可能仅因 γ 矩阵插入位置的交换而等价 (如 γ_5 在 u/d 间交换), 物理结果相同, 应归并为单一计算, 符号按 γ 转置性质传播。对应公式 (2)。

$$c_{\text{neighbor}} = c_{\text{cur}} \times \frac{rs_n}{rs_{\text{cur}}} \times \frac{vs_n[v_n]}{vs_{\text{cur}}[v_{\text{cur}}]} \quad (\text{跨 variant 图连通边传播符号}) \quad (2)$$

算法原理 (autowick.py:454-631):

1. **variant 生成** (generate_all_swaps, autowick.py:513-535): 对每个 γ 插入位置, 构造”交换该 γ 两侧夸克”的 2^n 种交换组合, 得到同一图的所有变体 (γ 交换对应 $\text{swap_sign}(G) = -T_{\text{sign}}(G)$, autowick.py:546-550)。
2. **并查集连通** (autowick.py:467-511): 以变体 (规范化排序后的 peram 条目列表) 为键建立哈希, 相同键的图合并——变体共享即等价连通。
3. **BFS 符号传播** (autowick.py:588-629): 在变体共享图上从代表元 BFS, 沿边按 (2) 传播相对系数, 正确处理传递性连通 (A-B-C 中 A 与 C 不直接共享变体的情况)。

核心算法

核心算法: 并查集 (路径压缩) + BFS (collections.deque); 变体数为 2^{n_γ} , n_gamma 为 γ 插入数; 图键哈希为 $\mathcal{O}(L)$ (L 为 peram 条目长度)。推断: 相比两两比对的全配对 $\mathcal{O}(N^2)$ 等价判定, 本实现按变体键一次哈希即得连通分量, 对 N 个图 V 个变体为 $\mathcal{O}(NV)$ 量级, 优势显著。

独特竞争力: 等价图归并直接减少后续 einsum 计算量 (run_wick_analysis 的 use_equivalence=True 时每等价类只计算一次并加权累加, dynamic.py:1135-1139 中 total_coeff 求和为 0 的条目直接跳过), 是性能优化与符号正确性双重保证。

2.3 C3 动态收缩工作流 (dynamic.py)

物理图像

物理图像: 高层工作流把 Wick 分析 (C1/C2) 与数值收缩解耦: 先注册 peram / 顶点 / γ 数据到三个注册表, 再生成”收缩计划” (plan), 最后按需逐图执行 einsum。对应公式 (3)。

$$C(t) = \sum_{i \in \text{plan}} \left(\sum_{(g,d,c) \in \text{equiv}_i} c \right) \cdot \text{cached_contract}(\text{ein}_i, \tau, \Gamma, V) \quad (3)$$

三注册表设计 (dynamic.py:88-330):

1. **PeramRegistry** (dynamic.py:88-177): 按 (味, 时间标签对) 注册传播子, resolve 时按组合类型 (如 'uu~d') 与时间自动选对应 peram; dtype 归一化。

2. `VRegistry` (dynamic.py:179-263): 按 (顶点名, 时间标签) 注册 VVV/VDV; `_per_region_v_names` (dynamic.py:331) 按时间端独立编号。
3. `GammaRegistry` (dynamic.py:265-330): 按名称注册 γ 矩阵; 支持 'Projector' 双侧投影序列 (dynamic.py:1166-1184)。

核心执行 `calculate_contraction` (dynamic.py:1076-1194): 先检查等价类系数和, 为 0 直接返回 (跳过计算); 解析 einsum 字符串、解析注册表张量 (`p_vars` / `v_vars` / `g_vars`); 支持 0index 覆盖输出指标与 Projection 双侧投影; 最后 `total_coeff * cached_contract(ein, *tensors)`。

核心算法

核心算法: 计划缓存 (`_plan_cache_key`, dynamic.py:439-460) —— 相同的算符组/前缀/等价开关组合直接命中缓存, 跳过重复 Wick 分析 (Wick 分析是纯符号计算, 缓存命中可省秒级开销); 收缩本身走 C5 的表达式缓存。 `validate_plan` (dynamic.py:1201-1245) 在实际计算前批量校验注册缺失, 避免运行到一半 `KeyError` 中断。

独特竞争力: 将”符号分析 (可缓存、与数据无关)”与”数值收缩 (需数据、按需执行)”彻底分层; 配套校验、投影、指标覆盖能力, 是工程上最完整的 Wick 收缩高层封装。

2.4 C4 Roofline 带宽分析器 (contractadviser.py)

物理图像

物理图像: 蒸馏收缩是大张量乘法, GPU 上常为带宽瓶颈 (读入中间张量耗时 $\gg$ 计算耗时)。用 Roofline 模型判断: 算术强度 $AI = \text{FLOPs}/\text{Bytes}$ 低于拐点 $\pi = P_{\text{peak}}/B_{\text{peak}}$ 即为带宽瓶颈, 对应公式 (4)。

$$\pi_{\text{dtype}} = \frac{P_{\text{peak}}(\text{dtype})}{B_{\text{peak}}} \quad (\text{FLOPs/byte 拐点}) \quad AI = \frac{\text{FLOPs}}{\text{bytes}_{\text{in+out}}} \implies \text{瓶颈} = \begin{cases} \text{带宽}, & AI < \pi \\ \text{计算}, & AI \geq \pi \end{cases} \quad (4)$$

算法原理 (contractadviser.py:1-15, 742-1194):

1. **硬件模型** (`HardwareSpec`, contractadviser.py:28-71): 按 dtype 选 FP16/FP32/FP64 峰值算力, 计入计算/带宽效率因子与 complex 修正 (`complex_compute_eff`); 8 个硬件预设 (contractadviser.py:81-120+).
2. **瓶颈判定:** 用 `opt_einsum` 路径分析估计 FLOPs 与最大中间张量, 结合每张量字节数计算算术强度, 与拐点比较分四级严重程度 (`none/mild/significant/severe`).
3. **切分建议** (`SlicingSuggestion`): 对自由指标逐一分析切块后的工作集大小与内存峰值, 给出建议块大小与理由 (README.md:1080-1142)。

核心算法

核心算法: Roofline 判定 + 切分收益扫描; 复杂度为 $\mathcal{O}(N_{\text{free}})$ 次张量大小评估。**未验证:** 硬件预设参数 (如 V100 FP64 7.8 TFLOPS、带宽 900 GB/s, contractadviser.py:84-90) 为产品规格, 未在实测环境复核。

独特竞争力: 面向蒸馏收缩定制 (大维自由指标、complex128、A100/A800/H20 中国特供硬件预设), 可预先规避带宽瓶颈、指导内存切分, 是同类工具 (如 opt_einsum 自身路径优化) 之上更高层的性能决策层。

2.5 C5 缓存 einsum 收缩入口 (cached_contract)

物理图像

物理图像: 蒸馏收缩中同一 einsum 表达式 (同字符串 + 同形状) 被反复调用 (如循环遍历 t_{src}), 路径编译 (contract_path) 是纯开销可一次性支付; 表达式缓存后每次直接执行。对应公式 (5)。

$$\text{key} = (\text{einsum_str}, \text{shapes}, \text{opt_key}) \longrightarrow \text{expr} \in \text{cache} \implies C = \text{expr}(\tau_1, \tau_2, \dots) \quad (5)$$

算法原理 (base_functions.py:347-441):

1. **热路径** (base_functions.py:375-390): 构造缓存键 (字符串 + 形状元组 + 策略键), 命中直接 `expr(*tensors)` 执行。
2. **冷路径** (base_functions.py:392-441): `optimize=True` 时依次尝试 `auto/greedy/optimal/dp` 四种策略, 以 `(flops, largest_intermediate)` 二元组择优 (先比 FLOPs 再比中间张量, base_functions.py:413-427); 用 `np.broadcast_to` 零内存占位符做形状校验与路径搜索 (`_validate_einsum_shapes`, base_functions.py:404-405); 成功后 `contract_expression` 编译并缓存。

核心算法

核心算法: 字典缓存 (键 = 表达式 + 形状 + 策略) + 多策略择优; 热路径为 $O(1)$ 查表。**推断:** Wick 循环场景下 (如 31 个例子脚本中的 peram 时间循环) 路径编译开销占比可达毫秒级/次, 缓存后降至微秒级; 实际加速比未测基准。

独特竞争力: 比裸 `opt_einsum.contract` 多出: 多策略自动择优 (裸调用默认 `greedy`)、形状错误提前诊断 (面向用户可读错误信息)、零内存校验 (占位符不占显存)、全局缓存清理 (`clear_cache`)。AGENTS.md 约定全包收缩统一走此入口。

2.6 C6-C11 浅析 (一段式概述)

C6 统计分析 (analyse.py): **Jackknife** (analyse.py:325-400) 单点消除重采样, $\text{sample}_k = -(\sum_{\text{all}} -\text{data}_k) / (N_{\text{conf}} - 1)$, 误差 $\sqrt{N_{\text{conf}} - 1} \times \text{std}$, 协方差用 `einsum('n...i,n...j->...ij')` 向

量化外积构建 (analyse.py:394); meff (analyse.py:521-569) 提供 $\log/\cosh/\text{GEVP}$ 三式 ($\frac{1}{a} \ln \frac{C(t)}{C(t+1)}$ 、 $\frac{1}{a} \text{arccosh} \frac{C(t+2)+C(t)}{2C(t+1)}$)，通过 ArraySlicer 通用切片实现后端无关。ratio_3pt (analyse.py:631) 与 solve_gevp (analyse.py:807) 支撑 3pt/2pt 比值与广义本征值提取。

C7 顶点函数 (eigvectors/vertex.py): Mom_VdV_sink_t 实现 $V_{ij}(p) = \sum_x e^{-ip \cdot x} \phi_i^\dagger(x) \phi_j(x)$, Mom_VVV_sink_t 实现 $V_{ijk}(p) = \sum_x e^{-ip \cdot x} \varepsilon_{abc} \phi_i^a \phi_j^b \phi_k^c$ (含 Levi-Civita 颜色缩并与 6 种颜色排列符号); VdV_sink_t_link 支持带规范链接的平行输运顶点 (空间方向 / 守恒流时间方向两种模式, README.md:419-470); create_omega_accelerate 构造 Ω 蒸馏权重张量用于方差缩减。

C8 算符共轭 (baroperator.py): conjugate_operator (baroperator.py:245) 自动识别介子 (2q) / 重子 (3q) / 多夸克结构, 计算 γ 链的 H/T/C 变换与 Fermi 符号; diquark_symmetry (baroperator.py:269) 给出 $(CT)^T = \eta(CT)$ 的 diquark 转置对称性符号。

C9 懒加载 API (__init__.py): __getattr__ (__init__.py:137-153) 首次访问属性时 import_module 到具体模块 (如 .analyse.analyse 而非 .analyse), 避免 mpi4py/ cupy/sympy 等重依赖被无条件加载; __dir__ 支持 tab 补全。

C10 MPI 并行 (mpi_init.py): get_mpi_tlist 时间片轮询分配 ($\text{local} = (\text{global} - \text{rank}) / \text{size}$ 模 N_t 回绕); get_mpi_data 支持 8 种传输模式 (Send/Gather/ TGather/Allgather/Bcast/Scatter/TScatter/Transport) 并自动处理 numpy/cupy 转换。

C11 DR 基 γ 矩阵 (constant/gamma_matrix.py): 18 种 4×4 复矩阵 (DeGrand-Rossi 基, 手征变体), 含正/负宇称投影 $\frac{1}{2}(\gamma_3 \gamma_1)(1 \pm \gamma_4)$; PFF_Mom_to_gamma_new 用 Levi-Civita 张量将动量分量映射为 γ 指标对 (投影形状因子)。

3 独特性与竞争力评估

4 关键片段与公式

4.1 核心代码片段 (lstinputlisting 直引)

```

1  def compute_fermion_sign(contraction_pairs: list) -> int:
2      """
3      计算 Wick 收缩的 Fermi 子符号。
4
5      核心思路:
6      1. 将所有被收缩的场按其在算符字符串中的位置排序。
7      2. 按排序后的顺序排列收缩对索引。
8      3. 计算逆序数 → 符号 = (-1)^逆序数。
9      """
10     indexed_positions = []
11     for pair_idx, pair in enumerate(contraction_pairs):
12         quark_op_pos = pair[0]
13         antiq_op_pos = pair[1]
14         indexed_positions.append((quark_op_pos, pair_idx))
15         indexed_positions.append((antiq_op_pos, pair_idx))
16
17     pair_indices = [x[0] for x in indexed_positions]
18     return (-1) ** count_inversions(pair_indices)
19 
```


独特点	对比基准	优势与证据
算符字符串 $\rightarrow$ einsum 全自动	传统手工 Wick 推导	多强子图易漏易错; 本实现符号化穷举 + Fermi 符号自动计算 (<code>autowick.py:156-224</code>)
等价图归并 + BFS 符号传播	两两图配对比较	按变体键哈希一次连通, $O(NV)$ 替代 $O(N^2)$; 符号经 (2) 传递保证正确累计 (<code>autowick.py:454-631</code>)
计划缓存 (符号层) + 表达式缓存 (数值层)	裸 <code>opt_einsum.contract</code>	热路径 $O(1)$ 查表; 多策略择优 (<code>auto/greedy/optimal/dp</code>); 零内存形状校验 (<code>base_functions.py:375-441</code>)
Roofline 带宽预测 + 切分建议	纯运行时性能调优	计算前可判定瓶颈并规划切分, 含 A100/H20 等 8 预设 (<code>contractadviser.py:81-120</code>)
numpy/cupy 双后端 + MPI	单后端实现	<code>set_backend</code> 全局切换; MPI 数据搬运自动类型转换 (<code>base/mpi_init.py</code>)

表 1: 独特性评估 (数据来源: 源码证据/推导)

```

20 def generate_valid_contractions(quark_pos: list) -> tuple:
21     """
22     按夸克味分组，为每味生成所有合法的一一配对（完美匹配），
23     再取各味之间的笛卡尔积组合。
24     返回（所有合法收缩图列表，收缩图总数）
25     """
26     flavor_quarks = {}
27     flavor_antiquarks = {}
28
29     for pos, qtype, label in quark_pos:
30         if '^d' in qtype:
31             base_flavor = qtype.replace('^d', '')
32             flavor_antiquarks.setdefault(base_flavor, []).append((pos, qtype, label))
33         else:
34             flavor_quarks.setdefault(qtype, []).append((pos, qtype, label))
35
36     flavor_matchings = []
37     for flavor in sorted(set(list(flavor_quarks.keys()) + list(flavor_antiquarks.keys()))):
38         qs = flavor_quarks.get(flavor, [])
39         aqs = flavor_antiquarks.get(flavor, [])
40
41         if len(qs) != len(aqs):
42             raise ValueError(
43                 f"Flavor '{flavor}' quark count({len(qs)}) "
44                 f"anti-quark count({len(aqs)}), cannot form full contraction")
45
46         if not qs:
47             continue
48
49         matchings = []
50         for perm in itertools.permutations(range(len(aqs))):
51             matching = []
52             for q_idx in range(len(qs)):
53                 aq_idx = perm[q_idx]
54                 matching.append((qs[q_idx], aqs[aq_idx]))
55             matchings.append(matching)
56
57         flavor_matchings.append(matchings)
58
59     if not flavor_matchings:
60         return [], 1
61
62     all_diagrams = []
63     for combo in itertools.product(*flavor_matchings):
64         diagram = []
65         for flavor_matching in combo:
66             diagram.extend(flavor_matching)
67         all_diagrams.append(diagram)
68
69     return all_diagrams, len(all_diagrams)

```

autowick.py:156-224——Fermi 符号（逆序数法）与按味分组的合法缩并穷举。

```

1 def calculate_contraction(entry, *,
2     peram_registry, v_registry, gamma_registry,

```

```

3         optimize='auto',
4         Projection=False):
5     """将 :func:`run_wick_analysis` 输出的一行解析为张量并执行收缩。
6
7     先检查 ``entry[0]`` 中所有等价图的系数之和 ``total_coeff``:
8     若为 0 则直接返回 ``0``，跳过计算；否则调用 :func:`cached_contract`
9     执行收缩并将结果乘以 ``total_coeff``。
10
11     Parameters
12     -----
13     entry : list
14         :func:`run_wick_analysis` 返回的 ``plan`` 中的一行，
15         格式 ``[equiv_list, contraction_idx, wick_dict, diag_idx,
16         proj_label]``。 ``proj_label`` 为可选的第 5 项。
17     peram_registry : PeramRegistry
18         已注册 peram 数据的注册表。
19     v_registry : VRegistry
20         已注册 V 张量数据的注册表。
21     gamma_registry : GammaRegistry
22         已注册 gamma 矩阵数据的注册表。
23     optimize : str, bool, or list of str, optional
24         传递给 :func:`cached_contract` 的优化策略，默认 ``'auto'``。
25     Projection : bool or str, optional
26         若为非 ``False``: 使用双侧投影。 ``gamma_registry`` 中的 ``'Projector'``
27         必须注册为两个投影算符组成的序列 ``[proj_sink, proj_src]``，
28         ``Projector[0]`` 收缩汇端自旋， ``Projector[1]`` 收缩源端自旋。
29         额外维度标签 (字符串直接作为标签; ``True`` 时自动从 ``entry[4]`` 读取)
30         为两个投影算符共享的领先轴 (位于旋量指标之外)。
31         输出指标顺序为 ``XY + 额外标签 + 原大写输出指标``，
32         X, Y 为投影引入的两个新自由指标。
33         若 plan 由显式 ``Oindex`` 生成 (``entry[6]`` 为 ``True``)，
34         则输出严格等于 ``Oindex``: 投影指标取同一字母互相收缩掉，
35         额外标签不在 ``Oindex`` 中时被求和。
36         默认 ``False`` (不投影)。
37
38     Returns
39     -----
40     ndarray or int
41         收缩结果乘以 ``total_coeff`` 后的张量。
42         ``total_coeff == 0`` 时返回 ``0``。
43
44     Raises
45     -----
46     RuntimeError
47         如果张量数与 einsum 逗号段数不一致。
48     KeyError
49         如果 peram / V / gamma 名称在注册表中未找到。
50
51     Examples
52     -----
53     >>> plan = run_wick_analysis([...], Cpt='2pt')
54     >>> for entry in plan:
55     ...     result = calculate_contraction(
56     ...         entry, peram_registry=peram_reg,
57     ...         v_registry=v_reg, gamma_registry=gamma_reg)

```

```

58     ...     accumulator += result
59     """
60     # 检查总系数, 为 0 则跳过
61     equiv_list = entry[0]
62     total_coeff = sum(coeff for _, _, coeff in equiv_list)
63     if abs(total_coeff) < 1e-12:
64         return complex(0)
65
66     _, _, wick, diag_idx, *_ = entry
67     _output_labels = entry[5] if len(entry) > 5 else ''
68     _oindex_given = entry[6] if len(entry) > 6 else False
69     ein = wick['result_idx'][diag_idx][0]
70     perams = wick['peram'][diag_idx]
71     v_info = wick['V']
72     g_info = wick['gamma_pos']
73
74     p_vars = [peram_registry.resolve(p[2], p[4]) for p in perams]
75     v_names = _per_region_v_names(v_info) # 转为按时间端独立编号
76     v_vars = [v_registry.resolve(vn, vt) for vn, vt in v_names]
77     g_vars = [gamma_registry.resolve(g[1]) for g in g_info]
78
79     tensors = p_vars + g_vars + v_vars
80
81     # Oindex 覆盖输出大写指标 (entry[5] 存在即重写, 允许 Oindex='')
82     if len(entry) > 5:
83         lhs, rhs = ein.split('->')
84         n_lower = 0
85         for ch in rhs:
86             if ch.islower(): n_lower += 1
87             else: break
88         spin_out = rhs[:n_lower]
89         ein = f'{lhs}->{spin_out}{_output_labels}'
90
91     # Projection: 双侧投影, Projector[0] 汇端自旋, [1] 源端自旋
92     if Projection is not False:
93         # 确定标签: True→从 entry 读取, str→直接使用
94         if Projection is True:
95             _proj_label = entry[4] if len(entry) > 4 else ''
96         else:
97             _proj_label = str(Projection)
98         proj = gamma_registry.resolve('Projector')
99         try:
100             n_proj = len(proj)
101         except TypeError:
102             n_proj = -1
103         if n_proj != 2:
104             raise ValueError(
105                 "Projector 必须为两个投影算符组成的序列 "
106                 "[proj_sink, proj_src]。")
107         ein = _build_projection_einsum(ein, _proj_label, _output_labels,
108                                     oindex_given=_oindex_given)
109         tensors = tensors + [proj[0], proj[1]]
110
111     # 纠错: 张量数与 einsum 的逗号段数一致
112     n_parts = len(ein.split('->')[0].split(','))

```

```

113     if len(tensors) != n_parts:
114         raise RuntimeError(
115             f'张量数({len(tensors)}) einsum段数({n_parts}): '
116             f'perams={len(perams)} gammas={len(g_info)} V={len(v_info)}')
117
118
119     return total_coeff * cached_contract(ein, *tensors, optimize=optimize)

```

dynamic.py:1076-1194——收缩计划执行：等价系数合并、Oindex 覆盖、双侧投影、缓存收缩。

```

1 def cached_contract(
2     einsum_str: str,
3     *tensors,
4     optimize: Union[str, bool, List[str]] = 'auto'
5 ):
6     """
7     执行带缓存的张量收缩，支持自动择优。
8
9     Parameters
10    -----
11    einsum_str : str
12        收缩字符串，例如 'ab,bc->ac'。
13    tensors : array_like
14        参与收缩的张量。
15    optimize : str, bool, or list of str
16        - 字符串：直接使用该策略。
17        - True：自动尝试预定义策略列表，选择代价最低者。
18        - 字符串列表：手动指定候选策略列表。
19
20    Returns
21    -----
22    result : array_like
23        收缩结果。
24    """
25    from opt_einsum import contract_expression, contract_path
26
27    shapes = tuple(t.shape for t in tensors)
28
29    # ---- 解析 optimize → opt_key (热路径，只为构造缓存键) ----
30    if isinstance(optimize, str):
31        opt_key = optimize
32    elif optimize is True:
33        opt_key = _AUTO_OPTIMIZERS
34    elif isinstance(optimize, list):
35        opt_key = tuple(optimize)
36    else:
37        raise TypeError(
38            f"optimize 参数类型应为 str, bool 或 list, 收到 {type(optimize)}")
39
40
41    key = (einsum_str, shapes, opt_key)
42    expr = _expr_cache.get(key)
43    if expr is not None:
44        return expr(*tensors)
45

```

```

46 # =====
47 # 缓存未命中 —— 以下为冷路径，仅首次调用时执行
48 # =====
49
50 # ---- 解析 candidate_opts (冷路径) ----
51 if isinstance(optimize, str):
52     candidate_opts = [optimize]
53 elif optimize is True:
54     candidate_opts = list(_AUTO_OPTIMIZERS)
55 else:
56     candidate_opts = optimize # list
57
58 # ---- 提前校验形状，给出具体诊断信息 ----
59 _validate_einsum_shapes(einsum_str, shapes)
60
61 # 创建零内存广播视图作为占位符
62 placeholders = [
63     np.broadcast_to(np.empty((), dtype=t.dtype), t.shape)
64     for t in tensors
65 ]
66
67 best_path = None
68 best_cost = (float('inf'), float('inf')) # (flops, size)
69
70 for opt in candidate_opts:
71     try:
72         path, path_info = contract_path(
73             einsum_str, *placeholders, optimize=opt
74         )
75         # 直接使用 path_info.opt_cost
76         flops = path_info.opt_cost
77         size = path_info.largest_intermediate
78         cost = (flops, size)
79         if cost < best_cost: # 先比较 flops, 再比较 size
80             best_cost = cost
81             best_path = path
82
83     except Exception:
84         continue
85
86 if best_path is None:
87     tried = ", ".join(candidate_opts)
88     raise RuntimeError(
89         f"尝试了以下优化策略 [{tried}]，均无法为给定表达式和形状"
90         f"找到可行的收缩路径。表达式: {einsum_str}, 形状: {shapes}"
91     )
92
93 expr = contract_expression(einsum_str, *shapes, optimize=best_path)
94 _expr_cache[key] = expr
95 return expr(*tensors)

```

base_functions.py:347-441——缓存 einsum：热路径查表 + 冷路径多策略择优 + 零内存占位符校验。

4.2 核心公式汇总

$$\text{Fermi 符号: } \varepsilon_\pi = (-1)^{\text{inv}(\pi)} \quad (6)$$

$$\text{VDV 顶点: } V_{ij}(p) = \sum_x e^{-ip \cdot x} \phi_i^\dagger(x) \phi_j(x) \quad (7)$$

$$\text{VVV 顶点: } V_{ijk}(p) = \sum_x e^{-ip \cdot x} \varepsilon_{abc} \phi_i^a(x) \phi_j^b(x) \phi_k^c(x) \quad (8)$$

$$\text{Jackknife: } \text{sample}_k = -\frac{\sum_{\text{all}} -\text{data}_k}{N_{\text{conf}} - 1}, \quad \sigma = \sqrt{N_{\text{conf}} - 1} \text{std} \quad (9)$$

$$\text{meff: } m_{\text{eff}}(t) = \frac{1}{a} \ln \frac{C(t)}{C(t+1)} \quad (\text{log 式}) \quad (10)$$

$$\text{seq_peram: } \tau \longrightarrow \gamma_5 \tau^\dagger \gamma_5 \quad (11)$$

公式 (6) 与 `compute_fermion_sign` (autowick.py:156-173) 一一对应; (7)/(8) 对应 `vertex.py` 顶点函数 (README.md:393-418); (9) 对应 `Jackknife` (analyse.py:325-400); (10) 对应 `meff log 式` (analyse.py:533-540); (11) 为 γ_5 时间反演变换 (seqperam.py:4)。

极限校验: (6) 在零缩并对 (空串) 时符号为 +1, 与真空期望一致; (7)/(8) 在 $p \rightarrow 0$ 时退化为蒸馏重叠矩阵 $V_{ij} = \sum_x \phi_i^\dagger \phi_j = \delta_{ij}$ (本征矢正交归一时), 量纲自洽; (10) 在 $t \rightarrow \infty$ 大时间极限给出基态质量 (信号区定义), 符合物理预期。

5 参考源清单

参考源	类型	内容/作用
<code>.../contraction/autowick.py:6-449</code>	仓库	Wick 收缩引擎 (C1)
<code>.../contraction/autowick.py:454-631</code>	仓库	等价图识别 (C2)
<code>.../contraction/autowick.py:636-1110</code>	仓库	缩并图可视化
<code>.../contraction/dynamic.py:88-330</code>	仓库	三注册表 (C3)
<code>.../contraction/dynamic.py:576-1075</code>	仓库	收缩计划生成与缓存
<code>.../contraction/dynamic.py:1076-1458</code>	仓库	计划执行/校验/高层类
<code>.../contraction/contractadviser.py:28-120</code>	仓库	硬件模型与预设 (C4)
<code>.../base/base_functions.py:347-459</code>	仓库	缓存 einsum (C5)
<code>.../base/backend.py:1-31</code>	仓库	双后端切换
<code>.../analyse/analyse.py:325-914</code>	仓库	Jackknife/meff/GEVP (C6)
<code>.../eigvectors/vertex.py:1-647</code>	仓库	VdV/VVV/规范链顶点 (C7)
<code>.../contraction/baroperator.py:1-273</code>	仓库	算符共轭 (C8)
<code>lqcddb/README.md:1-1465</code>	文档	API 全参考
<code>lqcddb/src/lqcddb/AGENTS.md:1-40</code>	文档	包约定
<code>.../examples/contraction.*.py</code>	示例	31 个实际用例

6 结论

结论

穷尽剖析结论：lqcddb 蒸馏收缩框架核心部分 12 项候选，其中 5 深挖（Wick 引擎 / 等价图识别 / 动态收缩 workflow / Roofline 分析器 / 缓存 einsum）、6 浅析（统计分析 / 顶点函数 / 算符共轭 / 懒加载 API / MPI 并行 / DR γ 基）、1 排除（旧扁平模块 C12）。最强竞争力：**算符字符串 $\rightarrow$ einsum 表达式的全自动 Wick 收缩引擎 + 等价图归并 + 双层缓存**——把格点 QCD 中最易错的手工环节（Wick 缩并、Fermi 符号、顶点分配）符号化、自动化、可缓存化，并以 Roofline 分析支撑大规模 GPU 收缩的理性优化。

局限与风险

局限与未验证项： 硬件预设参数 (C4) 为产品规格，未在实测环境复核； 未验证最大算符规模 (4pt 多强子) 的枚举耗时基准； 计划/表达式缓存加速比未做实测基准（推断为毫秒 $\rightarrow$ 微秒量级）； 旧模块 C12 未逐一 diff，仅以”同 API 重构版”排除（目录规模实测 function_contraction 5892 行 vs lqcddb 8835 行）； 所有结论为源码证据（确证）+ 推导（推断）两级，无运行期数值验证。